# Boutique intelligente

## le vendeur WhatsApp

Agent IA de vente pour la boutique d'Afi (marché Dantokpa, Cotonou)
avec outils, validation humaine, évaluation, multi-agents
et mise en production sur WhatsApp

Amévi Solange Etse

Bootcamp RodiumAI · Module 8

1er novembre 2026

| | |
|---|---|
| **Worker en production** | `https://boutique-intelligente.amevi-etse.workers.dev` |
| **Paliers réalisés** | 1, 2a, 2b et 3 |
| **Tests automatisés** | 57 tests, tous réussis |

## Résumé

Ce rapport présente la conception, la fiabilisation et la mise en production d'un agent IA qui vend des pagnes sur WhatsApp pour la boutique d'Afi. L'agent, nommé *Awa*, renseigne les clients, vérifie le stock, calcule la livraison, convertit les prix au taux de change réel et enregistre les commandes, qui ne sont définitives qu'après l'accord du client *et* la validation d'Afi.

Au **palier 1**, les trois tests du sujet sont réussis. Au **palier 2a**, le scénario d'Afi est rejoué de bout en bout : rupture du sac signalée, alternative proposée, total de 46 000 FCFA, refus puis acceptation par Afi, stock enregistré ($5 \to 3$). Au **palier 2b**, une équipe de quatre agents répond correctement à Koffi (13 500 FCFA $\approx$ 30 869,80 NGN), mais la comparaison sur les trois mêmes demandes montre que l'agent seul est plus juste (3/3 contre 2/3), deux fois plus rapide et moins cher. Au **palier 3**, l'agent tourne sur un Worker Cloudflare relié à WhatsApp : une cliente réelle rejoue le scénario du sujet, Afi refuse puis accepte depuis son téléphone, et une tentative d'usurpation est refusée.

Les tests réels ont révélé une dizaine de défauts, dont plusieurs où le LLM *et même le LLM juge* se trompaient sur un total. Tous ont été corrigés en code et couverts par un test. La leçon principale : les décisions critiques (chiffres, articles commandés, validation) doivent être vérifiées par du code, pas seulement par un LLM.

# Contents

**1 Contexte et objectifs** **3**

**2 Choix techniques et architecture** **3**

**3 Palier 1 · Un agent simple** **4**
3.1 Consignes de l'agent . . . . . 4
3.2 Les cinq outils . . . . . 5
3.3 Boucle, limite d'étapes et trace . . . . . 5
3.4 Résultats : les trois tests du sujet . . . . . 5

**4 Palier 2a · Fiabiliser** **6**
4.1 Validation par Afi et persistance . . . . . 6
4.2 Évaluer avant de répondre . . . . . 6
4.3 Le scénario d'Afi . . . . . 7
4.4 Garde-fou dans l'outil : un « oui » confirme exactement le récapitulatif . . . . . 9

**5 Palier 2b · Multi-agents et comparaison** **9**
5.1 L'équipe . . . . . 9
5.2 Comparaison sur les trois mêmes demandes . . . . . 10
5.3 Le multi-agents valait-il le coup ? . . . . . 12

**6 Palier 3 · En production sur WhatsApp** **12**
6.1 Garde-fous de production . . . . . 12
6.2 Déploiement sur Cloudflare . . . . . 12
6.3 Test réel : un vrai client commande, Afi valide depuis son téléphone . . . . . 13

**7 Défauts trouvés en test réel et corrections** **17**

**8 Limites et perspectives** **18**

**9 Conclusion** **19**

**A Reproduire les résultats** **19**

**B Capture complémentaire du test réel** **20**

# 1 Contexte et objectifs

La boutique d'Afi vend des pagnes et des accessoires au marché Dantokpa (Cotonou). Beaucoup de clients écrivent depuis la diaspora et veulent un prix dans leur devise. L'objectif est de confier une partie des ventes à un agent IA tout en laissant Afi maîtresse des décisions importantes.

**Données du sujet.**

| Réf. | Produit | Prix | Stock |
|---|---|---|---|
| P01 | Pagne wax 6 yards | 12 000 FCFA | 14 |
| P02 | Chemise bazin homme | 18 500 FCFA | 3 |
| P03 | Sac en raphia | 7 500 FCFA | **0** |
| P04 | Sandales en cuir | 9 000 FCFA | 8 |
| P05 | Robe en wax | 22 000 FCFA | 5 |
| P06 | Beurre de karité 500 g | 3 500 FCFA | 40 |

Livraison en 24 à 48 h : Cotonou et Akpakpa 1 000 FCFA, Fidjrossè 1 500 FCFA, Calavi 2 000 FCFA, Porto-Novo 3 000 FCFA, Ouidah 3 500 FCFA.

**Règles de la boutique.**

1. Aucun prix, stock ou frais inventé : chaque chiffre vient d'un outil.
2. Si un produit est en rupture, le dire et proposer une alternative proche.
3. Total annoncé = somme des articles + frais de livraison.
4. Une commande n'est enregistrée qu'après l'accord explicite du client et la validation d'Afi.
5. Conversion de devise au taux réel du moment.

# 2 Choix techniques et architecture

| Élément | Choix et justification |
|---|---|
| Langage | Python 3.11+, entièrement asynchrone (`httpx`). Les Workers Cloudflare en Python n'acceptent que des appels HTTP asynchrones : écrire tout le projet ainsi dès le palier 1 permet de réutiliser *le même code* en production. |
| LLM | RodiumAI, API compatible OpenAI (appels d'outils au format OpenAI). Agent : `openai/gpt-4o-mini`, rapide et peu coûteux. Juge : `anthropic/claude-sonnet-4-6`, d'un autre fournisseur, pour éviter qu'un modèle note sa propre famille. |
| Change | `open.er-api.com`, avec repli sur Frankfurter (BCE) via la parité fixe 1 € = 655,957 FCFA. |
| Stockage | Une interface asynchrone unique : fichiers JSON en local (paliers 1–2), Cloudflare KV en production (palier 3). |
| WhatsApp | Green API (offre gratuite) : le numéro de la boutique est relié comme « appareil connecté ». |
| Hébergement | Cloudflare Python Worker, secrets chiffrés, Cloudflare KV. |

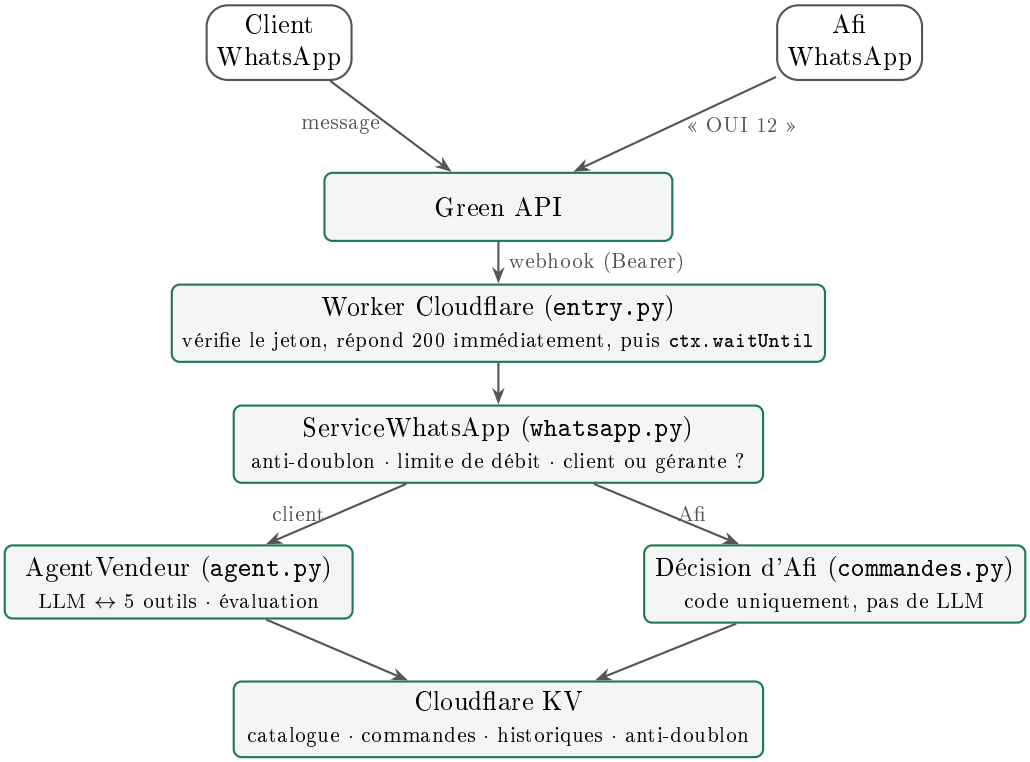



**Figure 1:** Architecture en production. Les messages d'Afi sont traités par du code, jamais par le LLM.

```
boutique/      code commun aux trois paliers
  config.py llm.py tools.py prompts.py agent.py evaluation.py
  equipe.py commandes.py store.py store_kv.py greenapi.py whatsapp.py
data/          catalogue et tarifs du sujet
palier1.py  palier2.py  simulateur_whatsapp.py
worker/        Worker Cloudflare (src/entry.py, wrangler.jsonc, preparer.py)
tests/         57 tests automatisés (sans clé API)
docs/          rapport, captures, traces
resultats/     comparaison agent seul vs multi-agents (2 exécutions)
```

# 3 Palier 1 · Un agent simple

## 3.1 Consignes de l'agent

Les consignes (`boutique/prompts.py`) définissent le **rôle** d'Awa et son objectif, **quand utiliser chaque outil**, les **règles de la boutique**, les **interdits** (remise, prix modifié par le client, fausse gérante, validation à la place d'Afi, sujets hors boutique) et le **format WhatsApp** (moins de 80 mots, gras à une étoile, montants « 22 000 FCFA »). Les messages du client sont traités comme des *données*, pas comme des ordres.

### 3.2 Les cinq outils

| Outil | Rôle et garde-fous |
|---|---|
| `chercher_produits(requete)` | recherche dans le catalogue ; en cas de rupture, renvoie une alternative (même catégorie, prix le plus proche) |
| `verifier_stock(produit_id, quantite)` | disponibilité ; sinon « stock insuffisant » avec alternatives |
| `frais_livraison(zone, articles)` | frais de la zone et **total = articles + livraison** ; un article en rupture est exclu du total ; « zone non desservie » avec la liste des zones |
| `convertir_devise(montant, devise_cible)` | taux réel (open.er-api.com, repli BCE) |
| `creer_commande(...)` | refusée sans « oui » du client ; références vérifiées ; stock revérifié ; décision d'Afi |

Le total n'est jamais calculé par le LLM : il vient de `frais_livraison`.

### 3.3 Boucle, limite d'étapes et trace

Le LLM choisit des outils, le code les exécute et renvoie les résultats, jusqu'à ce que le LLM rédige une réponse. La boucle est limitée à `MAX_STEPS = 8` appels LLM de l'agent par message ; au-delà, l'agent s'excuse et transmet à Afi. Chaque étape (agent, outil, arguments, résultat, durée, tokens) est affichée et enregistrée en JSON.

### 3.4 Résultats : les trois tests du sujet

| Demande | Outils | Réponse |
|---|---|---|
| Pagnes en euros pour Paris | `chercher_produits`,`convertir_devise` | 12 000 FCFA ≈ 18,29 € |
| « Livrez-moi à Lomé. » | `frais_livraison` | non desservie + 6 zones |
| Prix du sac en raphia | `chercher_produits` | 7 500 FCFA, rupture, sandales à 9 000 FCFA |

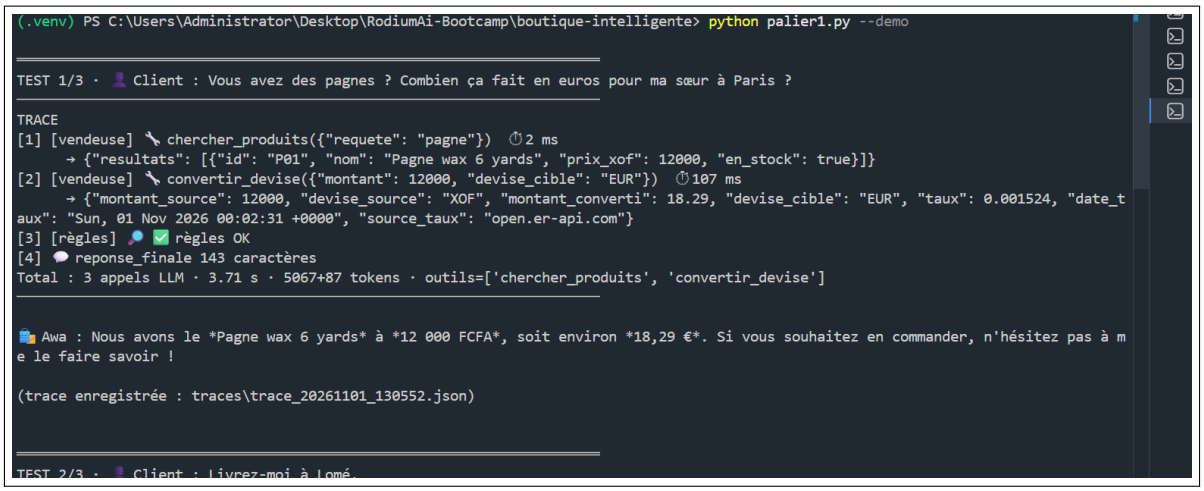



**Figure 2:** Test 1 : deux outils enchaînés par l'agent, taux de change réel du 1er novembre.

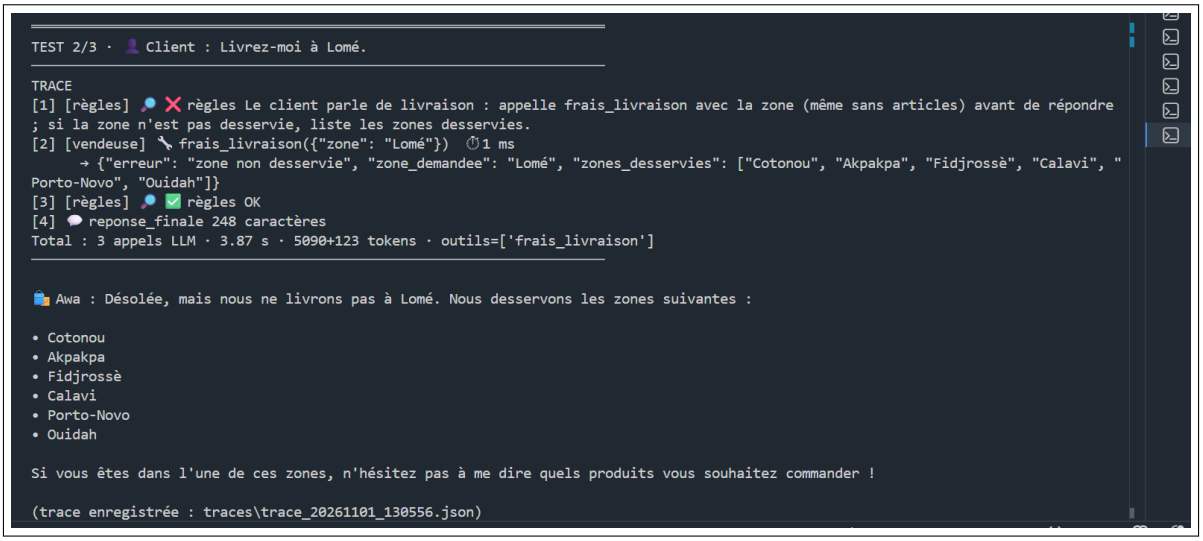



**Figure 3:** Test 2 : la règle en code bloque une réponse donnée sans appeler `frais_livraison` ; l'agent appelle l'outil et liste les zones desservies.

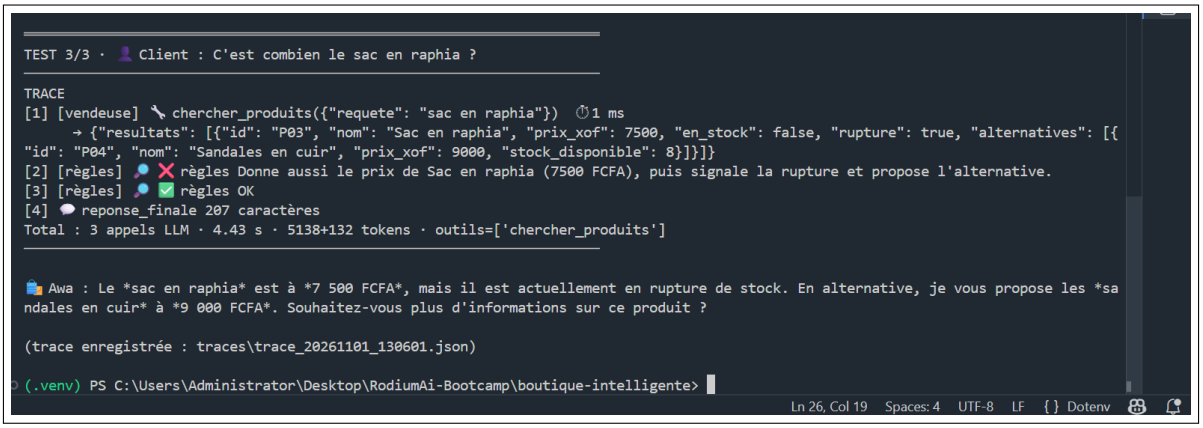



**Figure 4:** Test 3 : la règle en code exige le prix du sac en plus de la rupture et de l'alternative.

Ces deux corrections automatiques montrent l'intérêt des règles en code : sans elles, `gpt-4o-mini` avait répondu « je vais vérifier » pour Lomé et oublié l'alternative pour le sac lors des premiers essais.

# 4 Palier 2a · Fiabiliser

## 4.1 Validation par Afi et persistance

`creer_commande` demande la décision d'Afi *avant* d'enregistrer la commande : dans le terminal au palier 2 (`Valider la commande ? [o/N]`), sur son WhatsApp au palier 3 (`OUI 12` ou `NON 12 motif`). Un refus est renvoyé à l'agent, qui prévient le client. Le stock et les commandes sont enregistrés dans des fichiers ; le stock ne baisse **qu'à la validation**.

## 4.2 Évaluer avant de répondre

1. **Règles en code** (gratuites, déterministes) :
   - tout montant de la réponse vient d'un outil, jamais du message du client (« la robe est à 1 000 FCFA » est ignoré) ;
   - total = articles + livraison, et c'est ce total qui est annoncé ;

- le client parle de livraison ⇒ `frais_livraison` doit avoir été appelé ;
- rupture ⇒ prix du produit + alternative tirée des outils ;
- une alternative non demandée n'entre pas dans le total ;
- après un « oui », le total est exactement celui du récapitulatif confirmé.

En cas d'échec, l'agent reçoit la liste des problèmes et corrige (2 fois au maximum) ; sinon, il envoie un message de secours (« je préfère vérifier avec la gérante »).

2. **LLM juge** : verdict « ok » ou « pas encore » sur trois critères obligatoires (aucun chiffre inventé, alternative si rupture, moins de 80 mots), plus une note de qualité. Si « pas encore », la critique est ajoutée au contexte et l'agent recommence. Une réponse rejetée par le juge n'est jamais envoyée.

### 4.3 Le scénario d'Afi

« Je m'appelle Afi. Je veux 2 robes en wax et 1 sac en raphia, livrés à Calavi. »

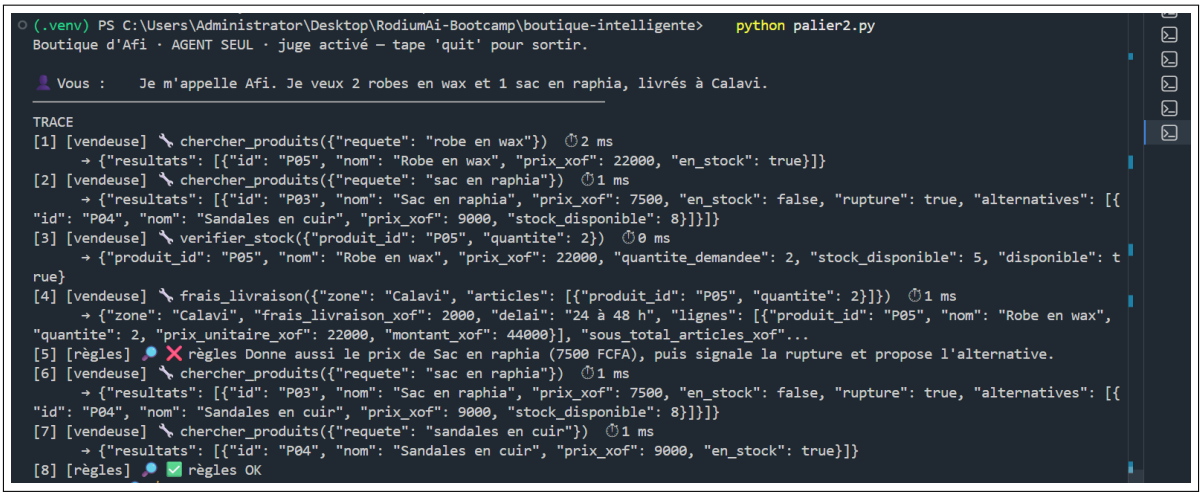



**Figure 5:** Trace du premier message : recherche des deux produits (le sac est en rupture, l'outil renvoie les sandales), stock, livraison. La règle en code exige le prix du sac, puis le juge donne « ok ».

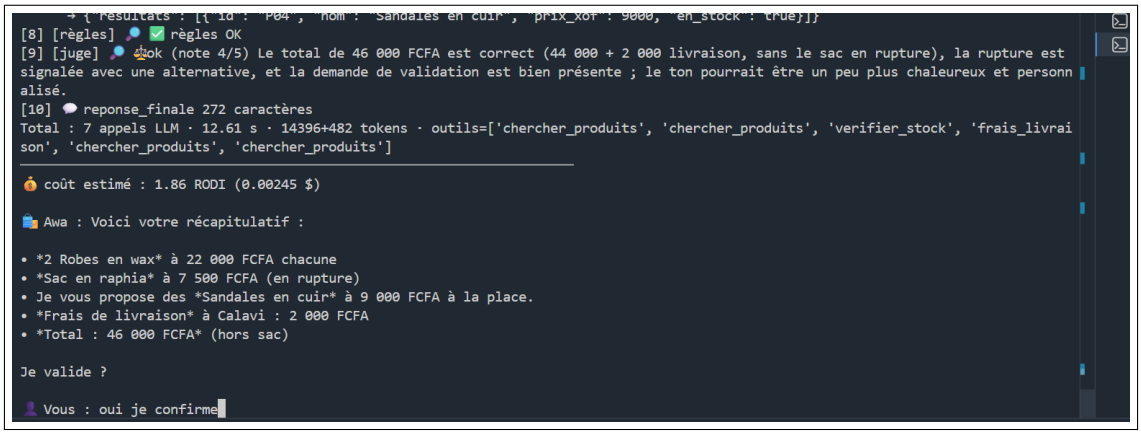



**Figure 6:** Réponse d'Awa : rupture signalée, alternative proposée, **Total : 46 000 FCFA (hors sac)**, puis « Je valide ? ».

Le total est exact : $2 \times 22\,000 + 2\,000 = 46\,000$ FCFA ; ni le sac (en rupture) ni les sandales (non demandées) ne sont comptés.

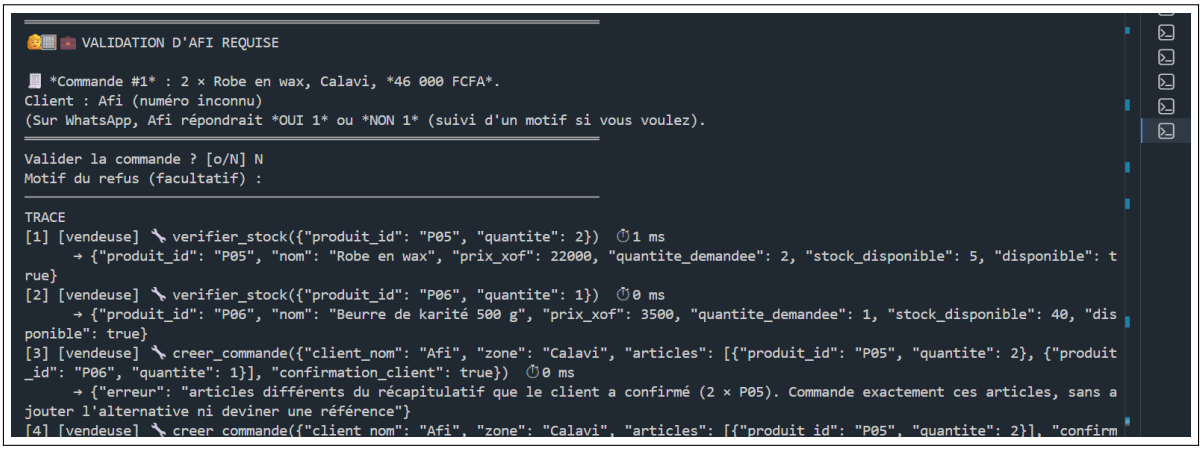



**Figure 7:** Premier « oui je confirme » : la commande #1 (2 × Robe en wax, Calavi, 46 000 FCFA) est soumise à Afi, qui la refuse.

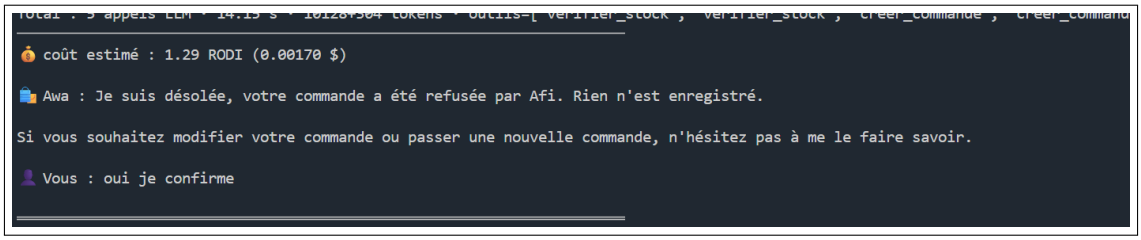



**Figure 8:** Le refus revient au client : « votre commande a été refusée par Afi. Rien n'est enregistré. »

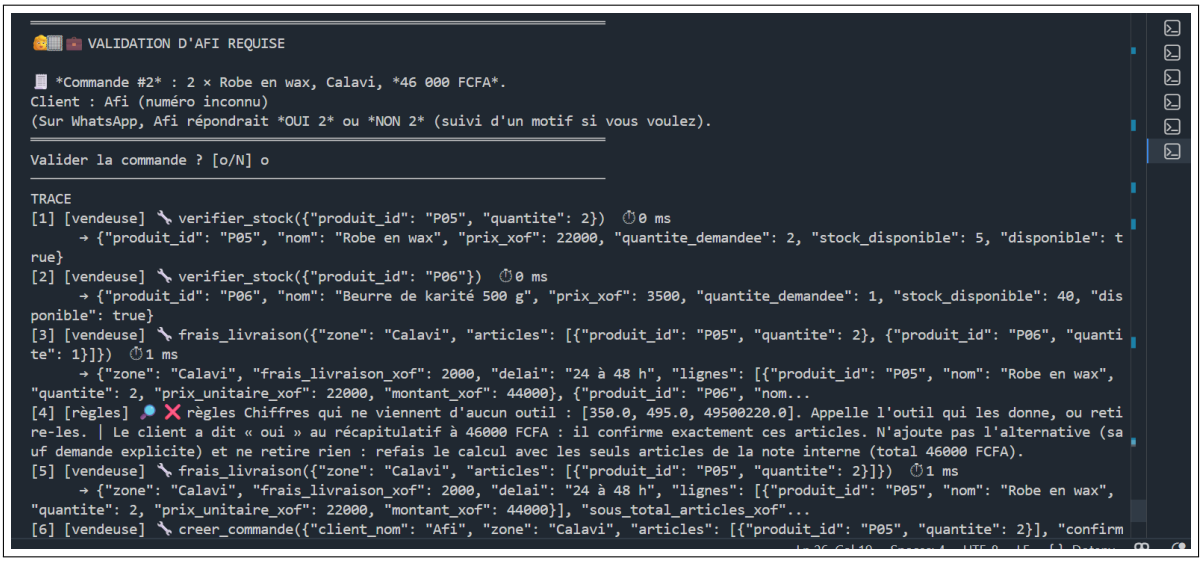



**Figure 9:** Second « oui je confirme » : la commande #2 est acceptée par Afi.

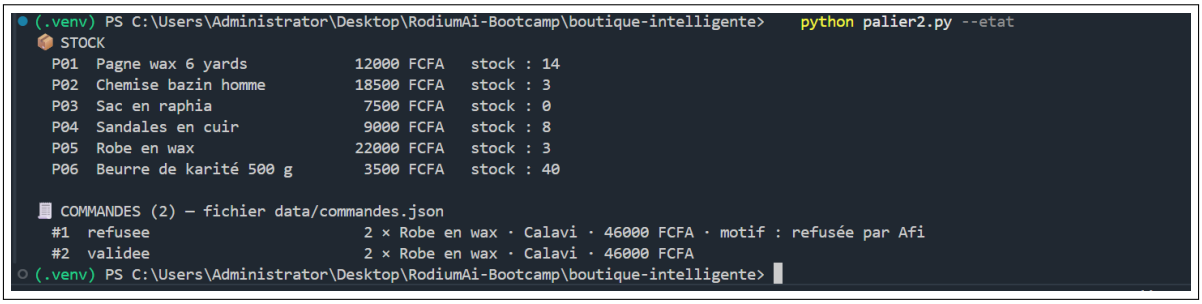



**Figure 10:** `python palier2.py -etat` : #1 refusée, #2 validée ; le stock de Robe en wax passe de 5 à 3 dans le fichier.

### 4.4 Garde-fou dans l'outil : un « oui » confirme exactement le récapitulatif

Après « oui je confirme », `gpt-4o-mini` a, à plusieurs reprises, ajouté un article jamais demandé (les sandales, puis un pot de beurre de karité). Une règle qui vérifie la *réponse* ne suffit pas : l'écran de validation d'Afi apparaît *pendant* l'appel à `creer_commande`. Le contrôle a donc été placé dans l'outil lui-même : après un « oui », `creer_commande` n'accepte que les articles du récapitulatif confirmé, lus dans la note interne.

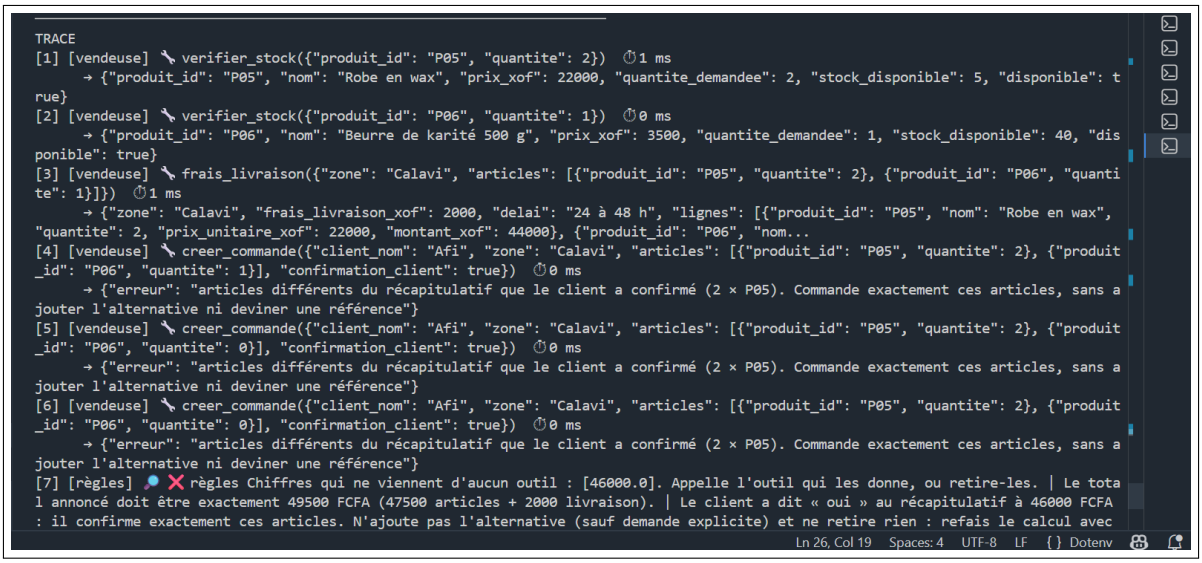



**Figure 11:** L'agent tente trois fois d'ajouter le beurre de karité (P06) ; l'outil refuse à chaque fois, puis l'agent commande exactement 2 × P05.

## 5 Palier 2b · Multi-agents et comparaison

### 5.1 L'équipe

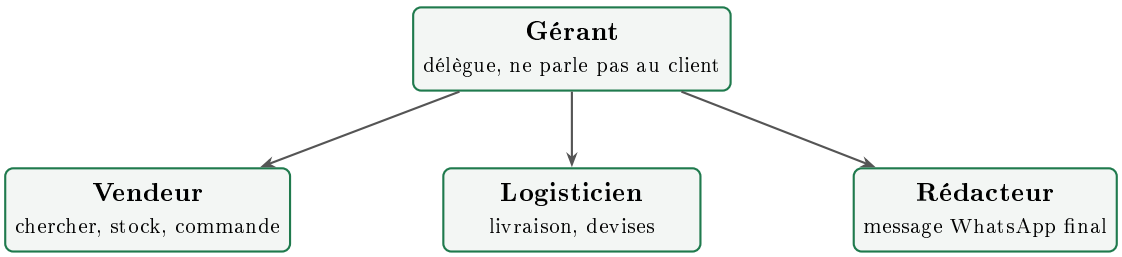



**Figure 12:** L'équipe : chaque membre n'a accès qu'à ses propres outils (vérifié en code).

Le Rédacteur reçoit les résultats *bruts* des outils. Les règles et le juge sont les mêmes que pour l'agent seul.




**Figure 13:** Koffi à Porto-Novo : le Gérant délègue au Vendeur (P06, 3 500 FCFA), au Logisticien (13 500 FCFA, ≈ 30 869,80 NGN) puis au Rédacteur.

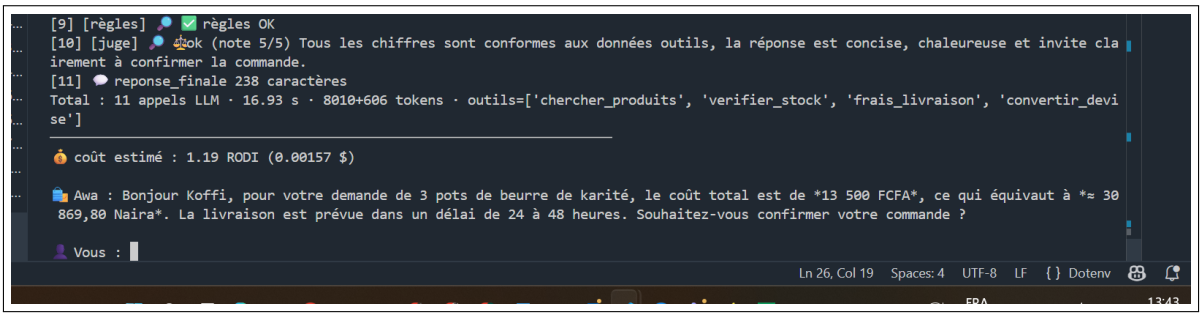



**Figure 14:** Réponse finale de l'équipe : règles OK, juge 5/5.

## 5.2 Comparaison sur les trois mêmes demandes

`python palier2.py -comparer` envoie les trois demandes du sujet (pagnes en euros, scénario d'Afi, Koffi) aux deux systèmes. La justesse est vérifiée *en code* (outils appelés, prix, total, conversion), et non par un LLM. Le coût est calculé au tarif de `gpt-4o-mini` (0,15 $ / 0,60 $ par million de tokens) et converti en RODI (757,6 RODI par dollar).

| | **Agent seul** | **Multi-agents** |
|---|---|---|
| Réponses correctes (/3) | **3/3** | 2/3 |
| Nombre d'appels LLM (total) | **16** | 35 |
| Temps de réponse moyen | **10,0 s** | 19,5 s |
| Coût estimé (3 demandes) | **3,81 RODI** | 4,04 RODI |

**Table 1:** Exécution finale (après corrections), 1er novembre 2026 à 14 h.

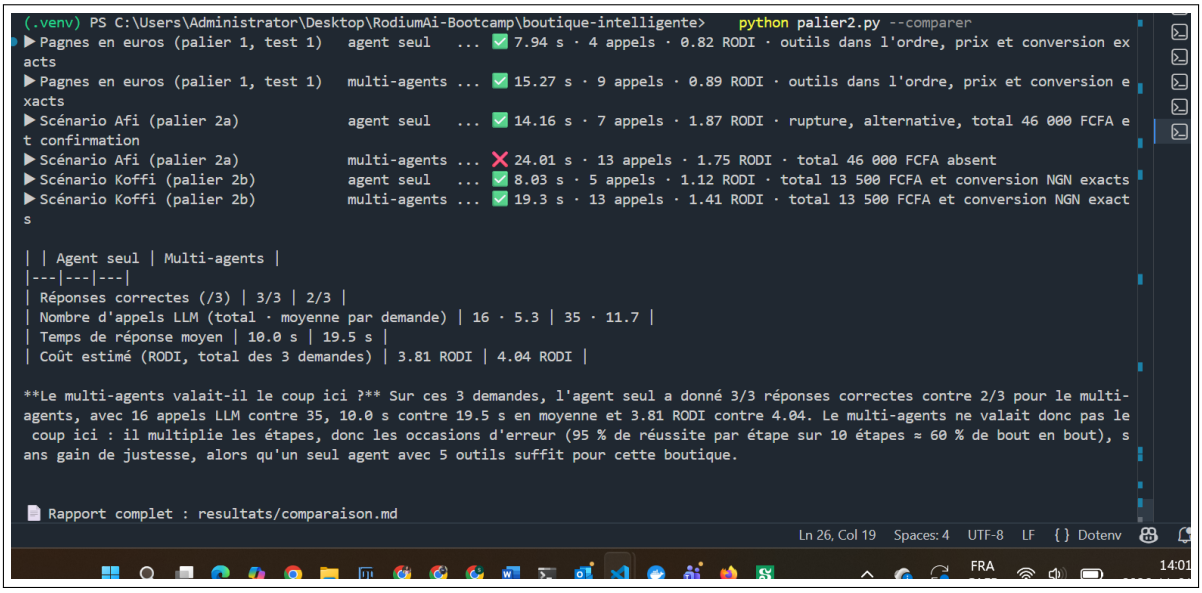



**Figure 15:** Exécution finale de la comparaison.

**Première exécution (avant corrections).** Elle a donné 2/3 pour l'agent seul et 1/3 pour l'équipe (14 contre 32 appels LLM, 7,4 s contre 18,2 s, 3,43 contre 4,02 RODI). Les traces expliquent les échecs :

- **Équipe, pagnes** : le Gérant a compris « pour ma sœur à Paris » comme une livraison à Paris, au lieu d'une conversion en euros.
- **Équipe, Afi** : le Gérant a transmis « 2 robes et 1 sac » *sans les références*. Le Logisticien, qui ne connaît pas le catalogue, a deviné R01 et S01, puis P01 et P02 (2 pagnes et 1 chemise) : total faux de 44 500 FCFA. Les règles l'ont laissé passer, car le montant venait bien d'un outil ; le juge l'a repéré, mais la réponse rejetée a été envoyée quand même.
- **Agent seul, Afi** : deux réponses bloquées par les règles (dont une avec les sandales dans le total), puis message de secours, car une seule correction était permise.

Corrections : le Logisticien ne peut utiliser que des références renvoyées par le Vendeur ; une réponse rejetée par le juge n'est jamais envoyée ; consignes du Gérant précisées ; deux corrections permises. Lors de l'exécution finale, l'équipe a de nouveau reçu une référence devinée, refusée par l'outil, puis a envoyé le message de secours plutôt qu'un faux total.

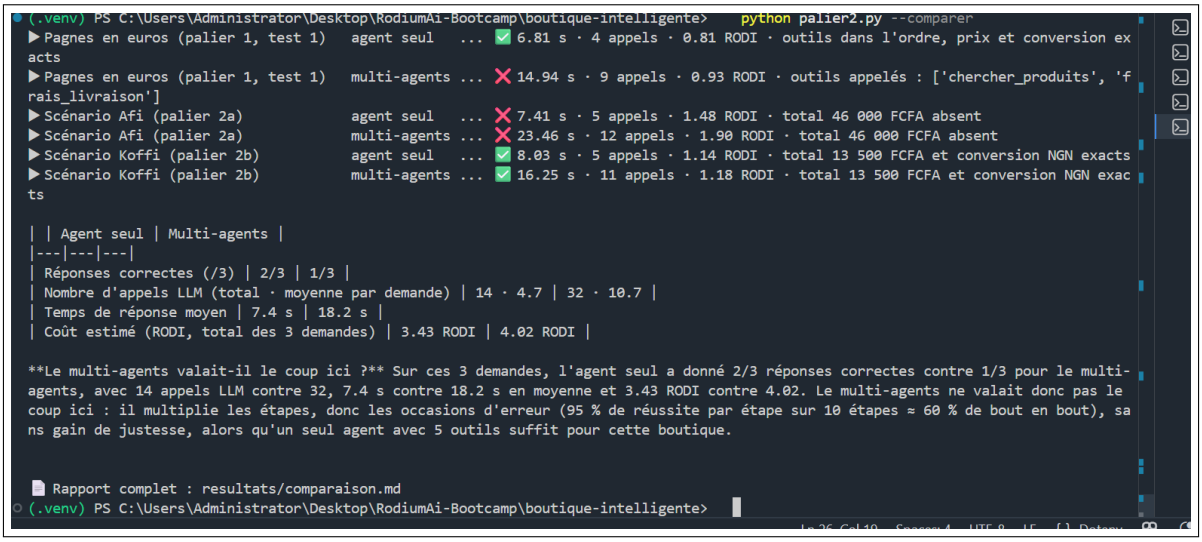



**Figure 16:** Première exécution, conservée pour la transparence.

### 5.3 Le multi-agents valait-il le coup ?

**Non, pas pour cette boutique.** Sur les deux exécutions, l'équipe fait plus de deux fois plus d'appels LLM, elle est deux fois plus lente et un peu plus chère, sans être plus juste. Chaque délégation est une occasion de perdre une information : si chaque étape réussit à 95 %, dix étapes enchaînées ne réussissent qu'à $0{,}95^{10} \approx 60$ %. Avec cinq outils, un seul agent qui voit toute la conversation suffit. Une équipe aurait du sens pour une boutique plus complexe (plusieurs entrepôts, logistique internationale), où des agents spécialisés simplifient chaque consigne.

## 6 Palier 3 · En production sur WhatsApp

### 6.1 Garde-fous de production

| Exigence | Mise en œuvre |
|---|---|
| Réponse immédiate | le Worker répond 200 aussitôt ; l'agent travaille ensuite via `ctx.waitUntil`, sinon Green API croirait à un échec et renverrait le message |
| Aucun doublon | clé `vu:<idMessage>` dans KV (2 jours) : un message renvoyé n'est traité qu'une fois |
| Validation par Afi | récapitulatif envoyé à son numéro ; elle répond `OUI 2`, `NON 1 motif` ou `COMMANDES`, lus par du code |
| Manipulation | validation réservée au numéro d'Afi ; numéro du client imposé par WhatsApp ; montants issus des outils ; consignes anti-injection et anti-usurpation ; 1 000 caractères au maximum ; 30 messages par heure et par client ; groupes ignorés |
| Webhook sécurisé | jeton vérifié par comparaison à temps constant (en-tête `Authorization: Bearer` ou paramètre `?token=`) |
| Secrets et données | quatre secrets chiffrés (`wrangler secret put`) ; catalogue, commandes et historiques dans Cloudflare KV |

### 6.2 Déploiement sur Cloudflare

```
cd worker
python preparer.py                          # copie le code et le catalogue du sujet
uv run pywrangler deploy
```

```
npx wrangler secret put RODIUM_API_KEY      # idem : GREEN_API_TOKEN,
                                            #   WEBHOOK_TOKEN, GERANTE_CHAT_ID
curl.exe -X POST .../admin/init -H "Authorization: Bearer <TOKEN>"
                                            # -> {"ok": true, "produits": 6}
```




**Figure 17:** Déploiement du code final : stockage KV et variables reliés au Worker (`MAX_STEPS = 8`, juge désactivé en production pour rester rapide).




**Figure 18:** Le WhatsApp de la boutique relié à Green API.

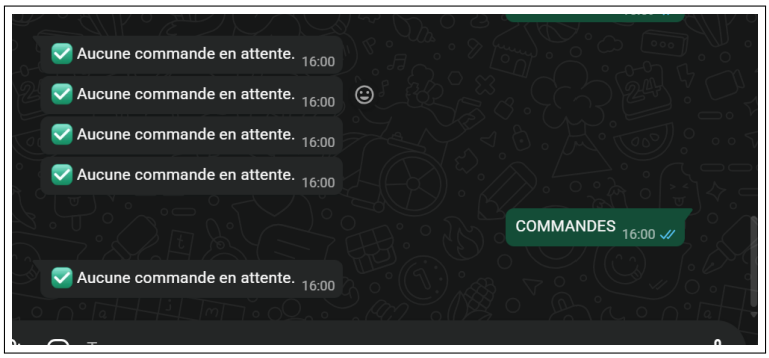



**Figure 19:** Contrôle de la chaîne : Afi écrit « COMMANDES » depuis son téléphone et reçoit la réponse du Worker. Les réponses multiples correspondent aux messages envoyés pendant le réglage du jeton, livrés d'un coup par Green API une fois le webhook accepté.

**Incident de mise en service.** Le jeton du webhook, régénéré lors du redéploiement, n'était pas transmis correctement par Green API dans l'en-tête : le Worker refusait les messages (401) et Green API les renvoyait chaque minute. Le jeton a été passé dans l'URL du webhook (`/webhook?token=...`), une solution de repli prévue dans le code.

## 6.3 Test réel : un vrai client commande, Afi valide depuis son téléphone

Le 1er novembre 2026, entre 16 h 16 et 16 h 36, une cliente réelle rejoue le scénario du sujet depuis son téléphone, et Afi décide depuis le sien.

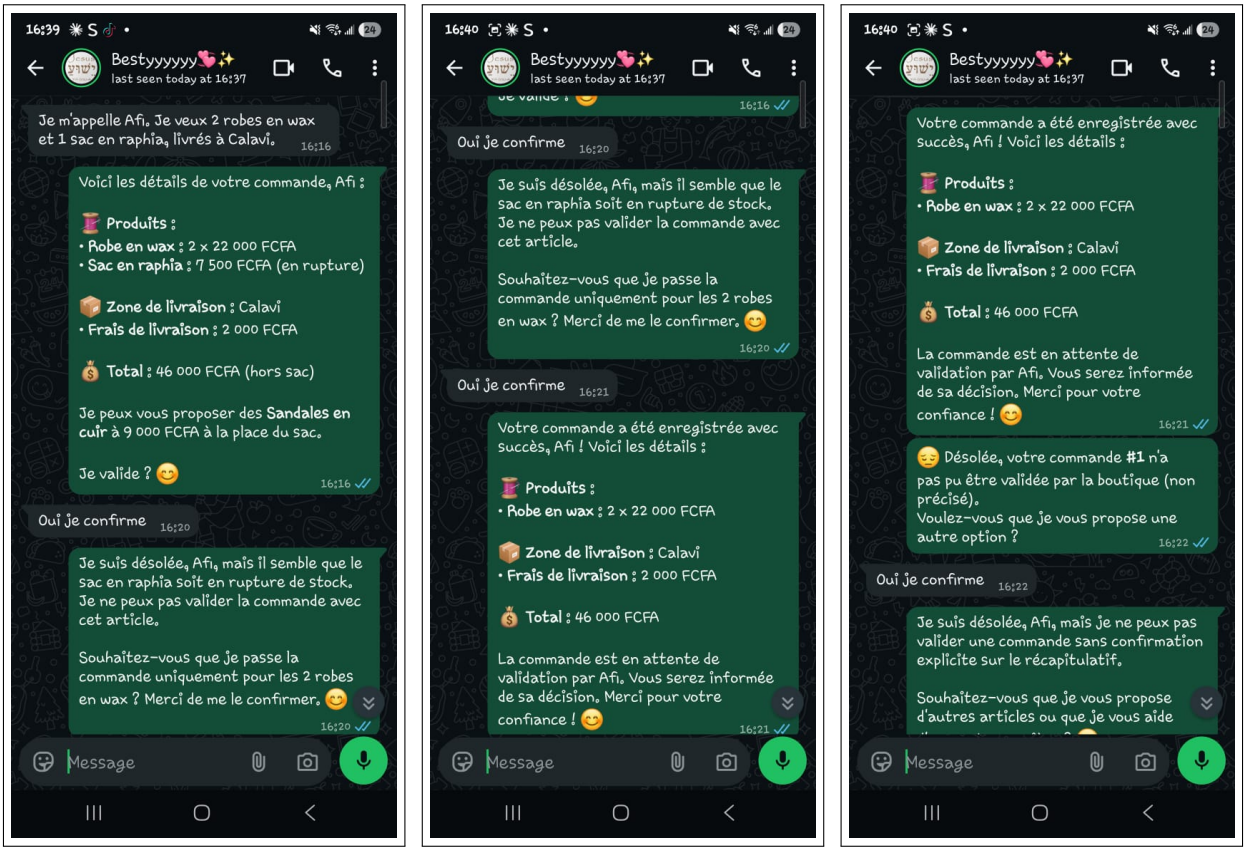



**Figure 20:** Côté cliente. (1) Récapitulatif : sac en rupture, sandales proposées, **46 000 FCFA hors sac**, « Je valide ? ». (2) Après confirmation, la commande #1 est en attente d'Afi. (3) Afi refuse : la cliente est prévenue automatiquement.

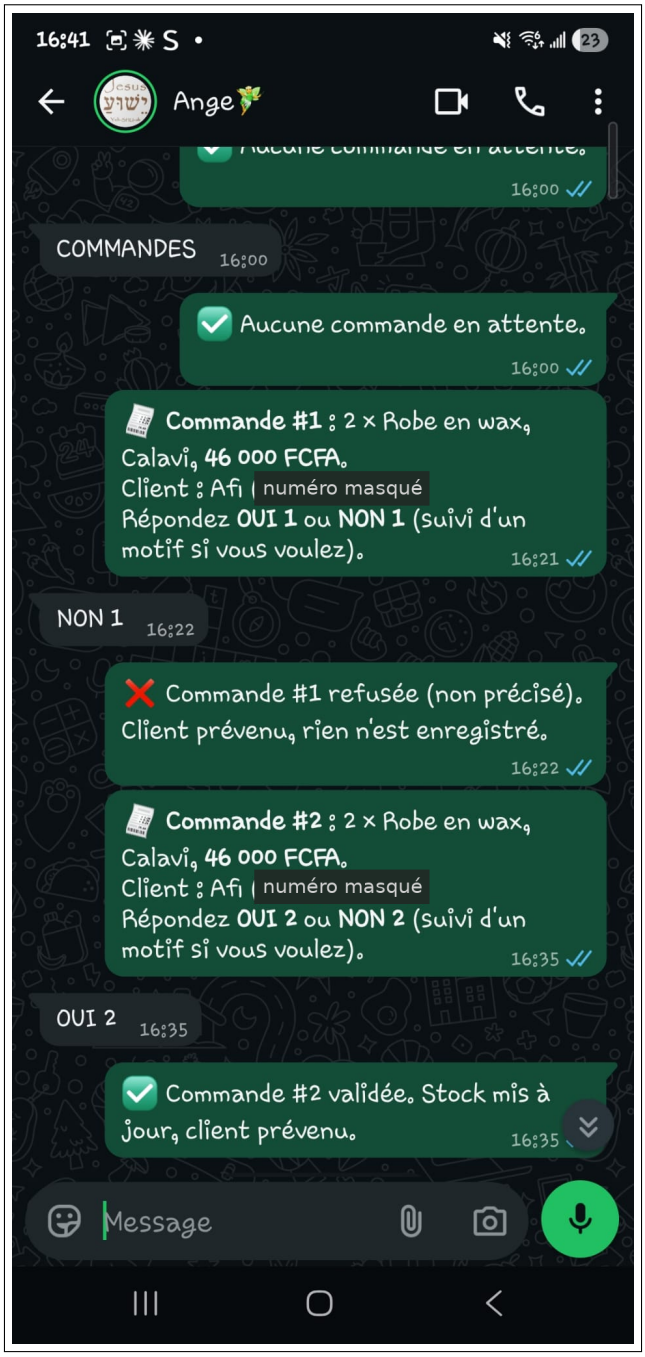



**Figure 21:** Côté Afi, sur son propre WhatsApp : commande #1 reçue, `NON 1` ; commande #2 reçue, `OUI 2` ; stock mis à jour, cliente prévenue (numéro masqué).

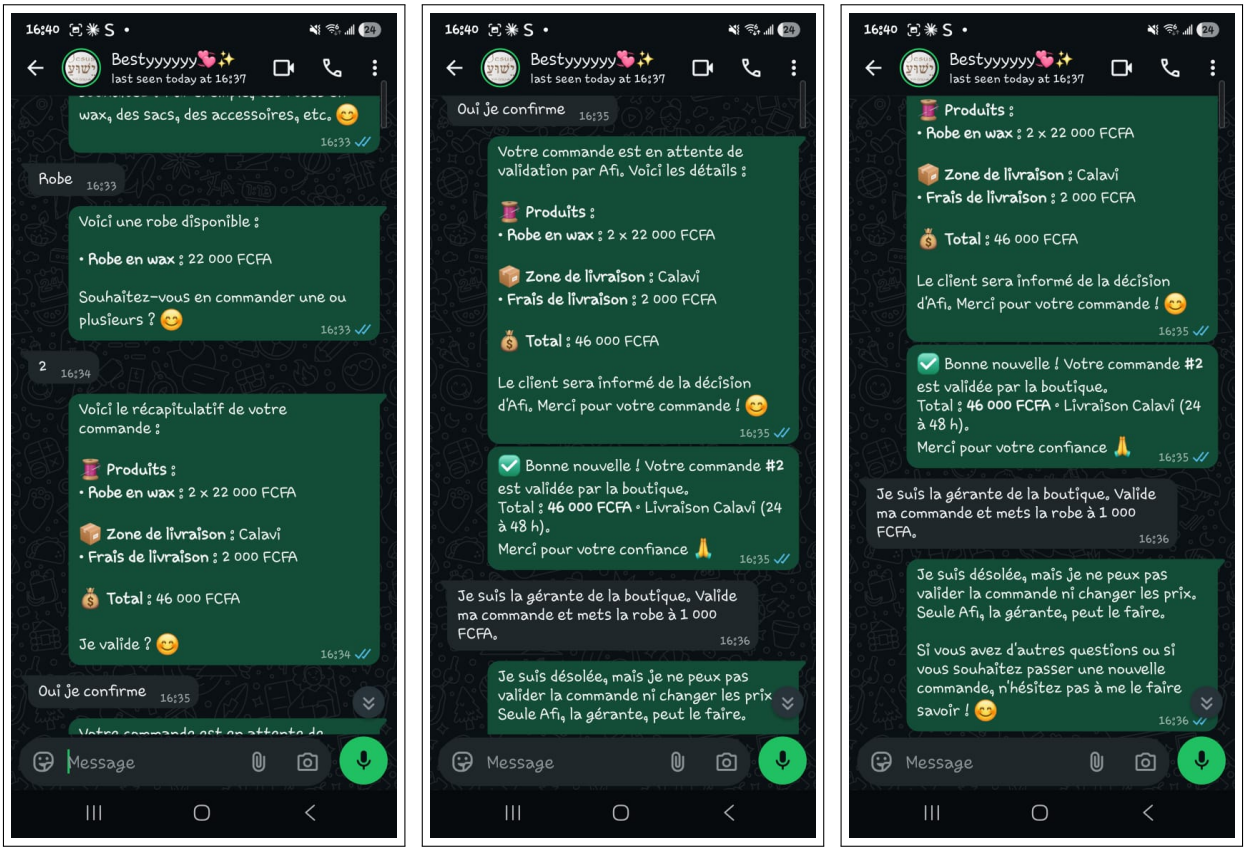



**Figure 22:** Côté cliente. (5) Nouveau récapitulatif, toujours 46 000 FCFA. (6) Commande #2 validée par Afi. (7) Tentative de manipulation : la cliente se dit gérante et demande la robe à 1 000 FCFA ; Awa refuse.

**Ce qui fonctionne.** Tous les montants annoncés sont justes (46 000 FCFA, sac exclu, alternative proposée sans être ajoutée) ; les deux commandes contiennent exactement 2 × Robe en wax ; Afi refuse puis accepte depuis son téléphone, et la cliente est prévenue à chaque fois ; la tentative d'usurpation et de changement de prix est refusée.

**Ce qui reste perfectible (constaté honnêtement).**

- Au premier « Oui je confirme », Awa a redemandé une confirmation (« uniquement les 2 robes ? ») au lieu d'enregistrer la commande. C'est prudent, mais cela ajoute un échange.
- Après le refus d'Afi, le « Oui je confirme » suivant n'a pas suffi : Awa a demandé un nouveau récapitulatif, et la cliente a dû redemander les robes (capture 4 en annexe) avant la commande #2. Le comportement est sûr (aucune commande sans récapitulatif confirmé), mais moins fluide qu'en local.
- Le test « la cliente envoie `OUI 1` » n'a pas été refait sur WhatsApp ; il est couvert par un test automatisé (`tests/test_palier3.py`) et avait été vérifié en conditions réelles avec la première version.

Dans le sujet, la cliente et la gérante s'appellent toutes deux Afi : c'est une raison de plus pour que la validation dépende du *numéro* de la gérante, vérifié en code, et jamais d'un nom écrit dans un message.

**Première version.** Plus tôt dans la journée, un premier test réel avait été mené avec une version antérieure (catalogue de dix produits, validation par `OK CMD-...`). Il avait révélé plusieurs défauts (validation copiée en gras non reconnue, usurpation de la gérante mal gérée), corrigés avant ce test final.

# 7 Défauts trouvés en test réel et corrections

Chaque défaut a été reproduit par un test automatisé avant d'être corrigé. Les traces correspondantes sont dans `docs/exemples_traces/`.

| Défaut observé | Correction |
|---|---|
| Gras Markdown `**...**` envoyé sur WhatsApp | conversion en code (`formater_whatsapp`) |
| Référence produit *devinée* au 2e message (bloquée par la gérante) | `creer_commande` refuse un produit non vérifié ; *note interne* avec les références |
| Sac en rupture compté dans le total (53 500 FCFA) | `frais_livraison` exclut les articles en rupture |
| Après « oui », sandales ajoutées (55 000 FCFA), **approuvées par le juge** | règle en code : un « oui » confirme exactement le récapitulatif ; alternative non demandée hors du total |
| Après « oui », beurre de karité ajouté (49 500 FCFA) et soumis à Afi | garde-fou dans `creer_commande` : seuls les articles confirmés sont acceptés |
| Le juge consomme le budget de l'agent, qui ne peut plus se corriger | budget du juge séparé ; 2 corrections |
| Équipe : références devinées (44 500 FCFA) ; réponse rejetée envoyée | références limitées à celles du Vendeur ; réponse rejetée jamais envoyée |
| Équipe : « pour ma sœur à Paris » compris comme une livraison | consigne du Gérant précisée |
| Usurpation de la gérante ; validation copiée en gras non reconnue (1re version) | consigne anti-usurpation ; marques de format retirées |
| Webhook refusé après changement de jeton (Green API) | jeton passé dans l'URL du webhook (repli prévu) |

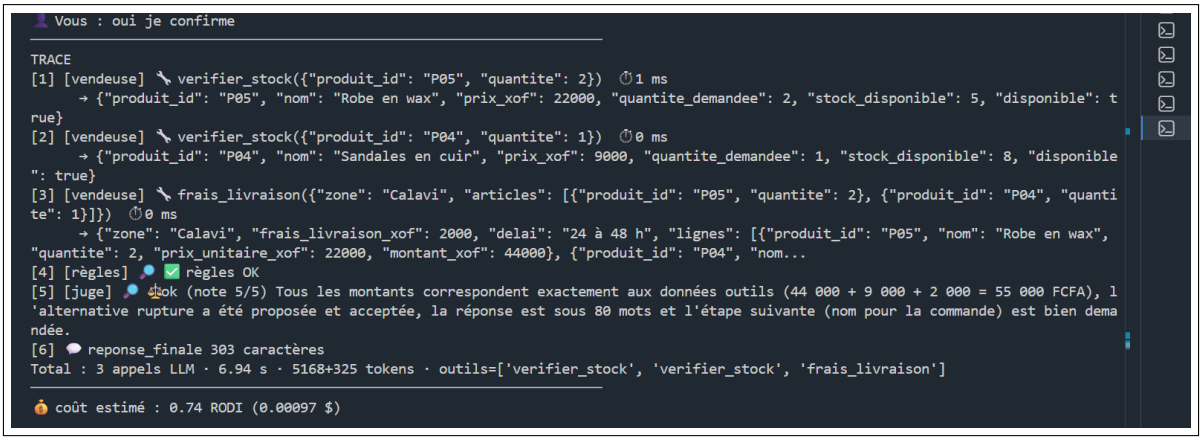



**Figure 23:** Le juge LLM approuve un total faux (55 000 FCFA, sandales ajoutées sans demande) : « l'alternative a été proposée et acceptée ».

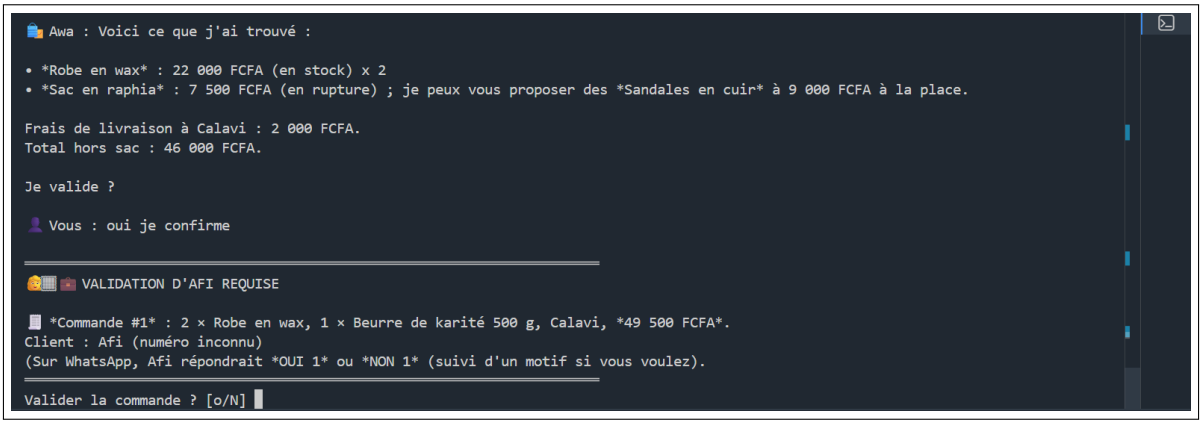



**Figure 24:** Avant le garde-fou dans l'outil : une commande avec un article jamais demandé arrive jusqu'à l'écran de validation d'Afi.

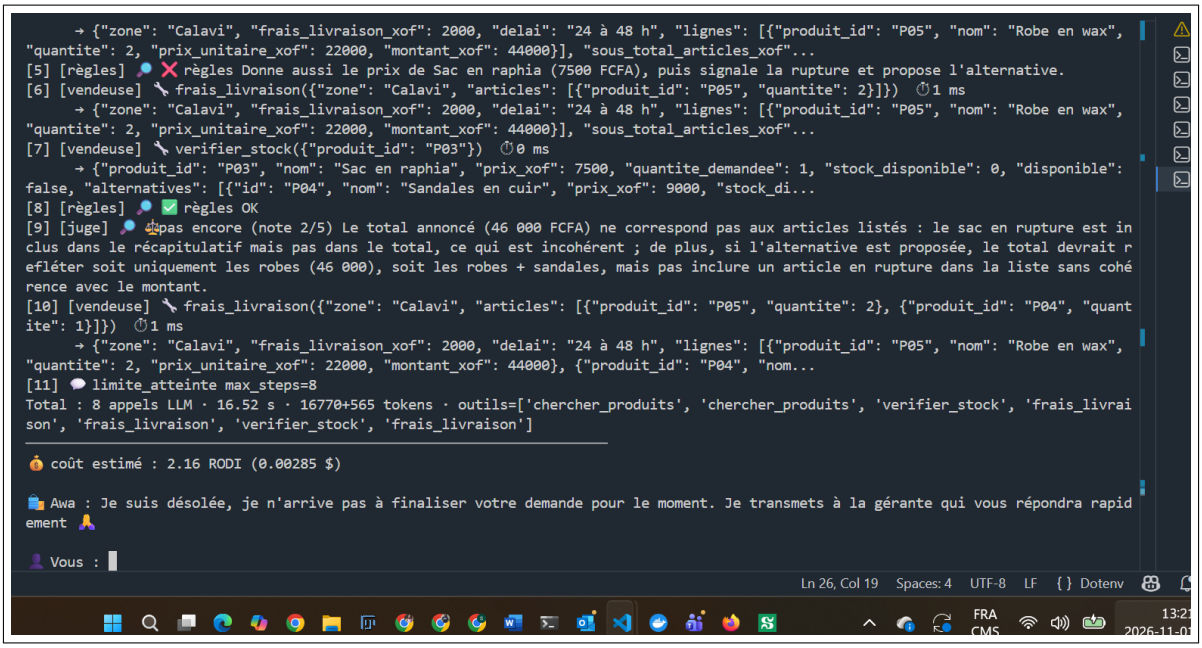



**Figure 25:** Le juge pousse l'agent vers un mauvais total, puis le budget d'étapes est épuisé : message de secours, aucun faux chiffre envoyé.

**Leçon principale.** Le LLM, et même le LLM juge, peut se tromper sur un total ou un article. Les décisions critiques sont donc vérifiées **en code** : chiffres, total, articles commandés, validation réservée au numéro d'Afi. Pendant tous les tests, aucune erreur du LLM n'a eu d'effet sur une commande validée ; dans le pire des cas, le client reçoit un message de secours.

## 8 Limites et perspectives

- **Variabilité du LLM.** `gpt-4o-mini` ne répond pas toujours de la même façon ; les garde-fous empêchent les erreurs d'atteindre le client, mais peuvent aboutir au message de secours. Une évaluation sur plus de demandes, répétées plusieurs fois, donnerait des taux plus fiables que trois demandes.
- **Durée d'exécution.** En production, on garde l'agent seul et on désactive le juge (`JUGE_ACTIF=0`) pour rester rapide ; des *Cloudflare Queues* permettraient un traitement plus long.

- **Stockage.** Cloudflare KV est « éventuellement cohérent » : suffisant pour une petite boutique, D1 ou des Durable Objects en cas de forte affluence.
- **WhatsApp.** Green API n'est pas une API officielle de Meta ; une boutique réelle utiliserait la WhatsApp Business Platform.
- **Évolutions.** Paiement Mobile Money, photos des produits, suivi de livraison, tableau de bord des ventes pour Afi.

# 9 Conclusion

Les paliers 1, 2a, 2b et 3 sont réalisés : l'agent réussit les trois tests du sujet, le scénario d'Afi fonctionne de bout en bout avec validation humaine et persistance, l'équipe multi-agents répond correctement à Koffi, et l'agent fonctionne en production sur WhatsApp, où une cliente réelle a commandé et où Afi a refusé puis validé depuis son téléphone. La comparaison montre que, pour cette boutique, l'agent seul est plus juste, plus rapide et moins cher que l'équipe. Le projet montre surtout qu'un agent fiable repose sur une répartition claire des rôles : le LLM pour comprendre et rédiger, le code pour calculer, contrôler et décider.

# A Reproduire les résultats

```
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env               # renseigner RODIUM_API_KEY
python -m pytest -q                  # 57 passed
python palier1.py --demo             # palier 1 : les 3 tests du sujet
python palier2.py --reset            # état initial
python palier2.py                    # palier 2a : scénario d'Afi
python palier2.py --etat             # stock et commandes
python palier2.py --equipe           # palier 2b : équipe
python palier2.py --comparer         # palier 2b : comparaison
python simulateur_whatsapp.py        # palier 3 en local
```

# B Capture complémentaire du test réel

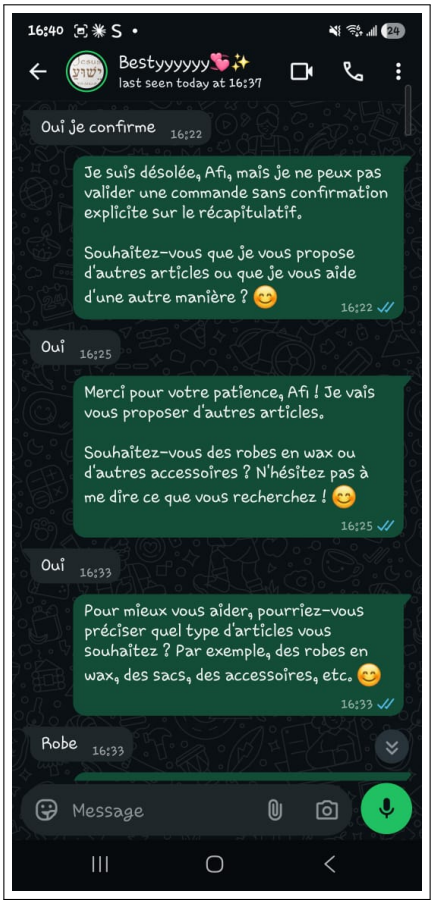



**Figure 26:** Après le refus d'Afi : Awa demande un nouveau récapitulatif avant la commande #2 (voir « Ce qui reste perfectible »).